

PORTADA

Arquitectura de Software

Semana 1 — Definiciones de Arquitectura de Software

Universidad de Antioquia · Ingeniería de Sistemas · 2554608 · 2026-2



APERTURA

Un proceso Rails para todo



CASO · FICHA TÉCNICA

Twitter, 2006–2010

Sistema	Twitter — red social de microblogging
Arquitectura original	Aplicación monolítica en Ruby on Rails, un solo proceso para todo
Síntoma público	La "Fail Whale" — página de error mostrada durante caídas frecuentes, 2007–2010
Crecimiento	De unos pocos miles de tweets/día en 2007 a cientos de millones años después
Respuesta	Rearquitectura progresiva: del monolito a servicios independientes sobre la JVM (2010–2013)
Fuente	Charlas públicas de ingeniería de Twitter (ej. Raffi Krikorian, "Twitter's Architecture") y su blog de ingeniería

CASO · QUÉ HABÍA DENTRO

Todo en un mismo proceso

La primera versión de Twitter atendía en un **único proceso Rails** tareas que hoy consideraríamos completamente distintas: recibir un tweet nuevo, calcular a quién mostrárselo, generar la línea de tiempo de cada usuario, enviar notificaciones, servir la página web.

Ninguna de esas tareas podía escalar, desplegarse o fallar por separado — si una se saturaba, arrastraba a todas las demás.

CASO · LO QUE NO ERA "UN BUG"

Decisiones de estructura, no de código

Las caídas repetidas de la "Fail Whale" no se resolvían corrigiendo líneas de código sueltas. El problema era **cómo estaba organizado el sistema completo**:

- Un solo componente concentraba responsabilidades que deberían estar separadas.
- No había forma de escalar "solo la generación de líneas de tiempo" sin escalar todo lo demás.
- Un fallo en una parte se propagaba a las demás — no había aislamiento.

Corregir esto exigía rediseñar la estructura del sistema, no parchar funciones.

CASO · LA REARQUITECTURA

De un proceso a varios servicios

Entre 2010 y 2013, los equipos de ingeniería de Twitter separaron el monolito en **servicios independientes** sobre la JVM (Scala/Java), cada uno con una responsabilidad clara — por ejemplo, un servicio dedicado a datos de usuario y otro a la generación de líneas de tiempo.

El comportamiento visible para el usuario cambió poco. Lo que cambió fue la estructura interna: cómo se dividían las responsabilidades y cómo se comunicaban entre sí.

TRANSICIÓN

¿Qué fue exactamente lo que cambió?

Nadie reescribió "el código de Twitter" desde cero. Lo que cambió fue algo más abstracto: **la forma en que las piezas del sistema estaban organizadas y relacionadas entre sí.**

A ese "algo más abstracto" es a lo que este curso llama arquitectura de software — y hoy le ponemos definiciones formales.

CONCEPTO · DEFINICIÓN 1

Perry & Wolf (1992)

Arquitectura de software = {Elementos, Formas, Restricciones}

- **Elementos:** las piezas del sistema — de proceso (componentes que hacen algo), de datos (lo que se procesa) y de conexión (lo que une a los otros dos).
- **Formas:** las propiedades y relaciones entre esos elementos.
- **Restricciones:** las reglas que limitan cómo pueden combinarse.

Una de las primeras definiciones formales del campo — plantea la arquitectura como una tripleta, no como un diagrama.

CONCEPTO · DEFINICIÓN 2

Garlan & Shaw (1994)

Cuando se diseñan y especifican sistemas de software grandes, la estructura del sistema en su conjunto plantea nuevos retos de diseño: la organización como conjunto de componentes, sus conexiones y patrones de interacción.

Es la definición que introduce el vocabulario que usaremos durante todo el curso: **componentes** (unidades con una función) y **conectores** (los mecanismos que permiten que se comuniquen).

En el caso Twitter: cada servicio nuevo es un componente; las llamadas entre servicios son los conectores.

CONCEPTO · DEFINICIÓN 3

Bass, Clements & Kazman (SEI)

La arquitectura de software de un sistema es el conjunto de estructuras necesarias para razonar sobre el sistema, que comprende elementos de software, las relaciones entre ellos, y las propiedades de ambos.

Esta definición, del *Software Engineering Institute*, agrega algo clave: la arquitectura existe para **razonar** sobre el sistema — tomar decisiones, predecir su comportamiento, comunicarlo a otros — no solo para describirlo.

CONCEPTO · DEFINICIÓN 4

ISO/IEC/IEEE 42010

El estándar internacional de descripción arquitectónica define arquitectura como:

Los conceptos o propiedades fundamentales de un sistema en su entorno, encarnados en sus elementos, relaciones, y en los principios de su diseño y evolución.

Novedad de esta definición: incluye explícitamente la *evolución* — la arquitectura no es solo cómo nace el sistema, sino los principios que guían cómo cambia con el tiempo.

CONTRAEJEMPLO

Mito: "la arquitectura es el diagrama"

Es común confundir la arquitectura con su representación gráfica: el diagrama de cajas y flechas que se dibuja en una herramienta.

El diagrama es una vista de la arquitectura, no la arquitectura misma. Twitter tuvo arquitectura —buena o mala— incluso en 2007, cuando nadie había dibujado un diagrama formal: la arquitectura es la estructura real que existe en el sistema en ejecución, se haya documentado o no.

CONCEPTO

¿Qué hace que una decisión sea "arquitectónica"?

No toda decisión de programación es una decisión de arquitectura. Una decisión es **arquitectónica** cuando cumple varias de estas condiciones:

- Es **costosa o difícil de revertir** una vez tomada.
- Afecta a **múltiples componentes** del sistema, no a uno aislado.
- Compromete **atributos de calidad** del sistema completo: rendimiento, escalabilidad, seguridad, mantenibilidad.
- Restringe las decisiones que se pueden tomar después.

Una decisión de **diseño de detalle**, en cambio, es local, barata de cambiar y no compromete al resto del sistema.

CONCEPTO · APLICADO AL CASO

Arquitectónico vs. detalle, en Twitter

Decisión	¿Arquitectónica?	Por qué
Separar la generación de líneas de tiempo en un servicio propio	Sí	Afecta a todo el sistema, es costosa de revertir, cambia cómo escala
Elegir el nombre de una variable interna de ese servicio	No	Local, sin impacto fuera de esa función, trivial de cambiar
Migrar de un proceso monolítico a servicios sobre la JVM	Sí	Redefine cómo se comunican, despliegan y fallan todos los componentes

TENSIÓN CONCEPTUAL

¿Arquitectura de software, o de la organización?

Separar Twitter en servicios independientes no solo cambió el código: también cambió **cómo se organizaban los equipos** — cada servicio terminó teniendo un equipo responsable.

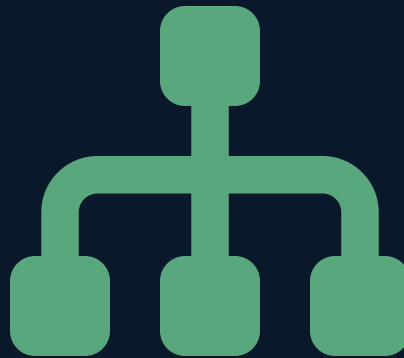
¿**La estructura del sistema determinó la estructura de los equipos, o fue al revés?** Esta pregunta tiene nombre propio en la disciplina.

CONCEPTO

La Ley de Conway (1967)

Las organizaciones que diseñan sistemas están limitadas a producir diseños que son copias de las estructuras de comunicación de esas organizaciones.

En otras palabras: **si tres equipos se comunican poco entre sí, es muy probable que construyan tres módulos poco integrados** — la estructura social se filtra, casi siempre sin querer, en la estructura técnica.



INTERACCIÓN

¿Arquitectónica o de detalle?

Clasifiquen cada decisión y justifiquen por qué:

1. Usar PostgreSQL en lugar de MySQL como base de datos del proyecto de Fábrica Escuela.
2. Nombrar un método `calcularTotal()` en vez de `total()` .
3. Exponer la lógica de negocio como una API REST en vez de embebida en el frontend.
4. Cambiar el color de un botón de "enviar".

5 minutos · usen los cuatro criterios de la diapositiva de "¿qué hace que una decisión sea arquitectónica?"

CONCEPTO EN DISPUTA

¿Toda la arquitectura se diseña por adelantado?

Postura clásica (Big Design Up Front): la arquitectura se define completa antes de escribir la primera línea de código de negocio.

Postura ágil/evolutiva: Twitter no diseñó en 2006 la arquitectura de servicios que tuvo en 2013 — la arquitectura **emergió y se transformó** a medida que el sistema creció, con decisiones tomadas cuando hubo evidencia real de que el diseño original no aguantaba.

No hay consenso único en la industria sobre cuánto diseñar por adelantado y cuánto dejar evolucionar — depende del riesgo, el dominio y qué tan cara es la reversión de cada decisión.

CONCEPTO

Por qué importa, en este curso

La arquitectura no es un ejercicio académico separado del código: es lo que determina si el sistema del proyecto de Fábrica Escuela va a **aguantar cambios, crecer y mantenerse** sin reescribirse cada sprint.

- Una mala decisión arquitectónica en el Sprint 1 puede ser muy costosa de corregir en el Sprint 3.
- Las próximas semanas de la Unidad 1 dan el vocabulario (roles, requisitos no funcionales, conectores) para tomar esas decisiones con criterio, no por intuición.

Por eso la frase que abre este sitio: "arquitectura antes que sintaxis".

SÍNTESIS

Ideas clave de hoy

1. La arquitectura es la **estructura** del sistema —elementos, relaciones y principios de diseño y evolución— no un diagrama ni un documento.
2. Cuatro definiciones formales (Perry & Wolf, Garlan & Shaw, SEI, ISO 42010) coinciden en algo: elementos + relaciones + razonamiento sobre el sistema.
3. Una decisión es **arquitectónica** si es costosa de revertir, afecta a varios componentes o compromete atributos de calidad.
4. La **Ley de Conway** conecta la estructura del sistema con la estructura de quienes lo construyen.
5. No hay consenso sobre diseñar toda la arquitectura por adelantado — Twitter es evidencia de arquitectura que evolucionó bajo presión real.

INSTRUCCIONES

Taller de hoy — Leyendo arquitectura en el ejemplo del martes

Demostración guiada por el docente — 30 a 40 minutos

El docente retoma el mismo ejemplo de la sesión anterior (backend en **Spring Boot** + frontend en **HTML/JavaScript**) y, en vivo, señala sobre ese mismo código cuáles decisiones son arquitectónicas y cuáles son de diseño de detalle, aplicando los cuatro criterios de la diapositiva 13.

INSTRUCCIONES · EJEMPLO COMENTADO

El mismo código, leído como arquitectura

```
// Decisión ARQUITECTÓNICA: separar el backend
// como servicio REST independiente del frontend.
// Afecta a todo el sistema y es costosa de revertir.
@RestController
@RequestMapping("/api/saludo")
public class SaludoController {

    // Decisión de DISEÑO DE DETALLE: el nombre del método
    // y el tipo de colección usados aquí son locales
    // y triviales de cambiar sin afectar al resto.
    @GetMapping
    public Map<String, String> obtenerSaludo() {
        return Map.of("mensaje", "Hola desde el backend");
    }
}
```

La estructura general (servicio separado, comunicación por HTTP/JSON) es arquitectura. Los nombres internos y el tipo de dato exacto usado no lo son.

No hay entregable de estudiantes en este taller: el objetivo es poder señalar, en cualquier código propio, qué parte es arquitectónica.

CIERRE

Para la próxima semana

- Revisar, aunque sea por encima, qué tecnologías usaba el software antes de la web (mainframes, cliente/servidor) — se retoma en "Historia de la Arquitectura de Software".
- Traer identificada una decisión de su propio proyecto de Fábrica Escuela que crean que ya es arquitectónica, según los criterios de hoy.

La semana 2 recorre la historia de la disciplina: de los mainframes al software como servicio.

